

Hardware-in-the-Loop Validation of Closed-Loop Field-Oriented Control

Designed and Implemented by : Afraaz Khan (afraazalig@gmail.com)

Overview:

This project implements a **Hardware-in-the-Loop (HIL) simulation framework** for validating Field-Oriented Control (FOC) algorithms on an **STM32G4 microcontroller** using **MATLAB/Simulink** as the plant and motor behavior simulator.

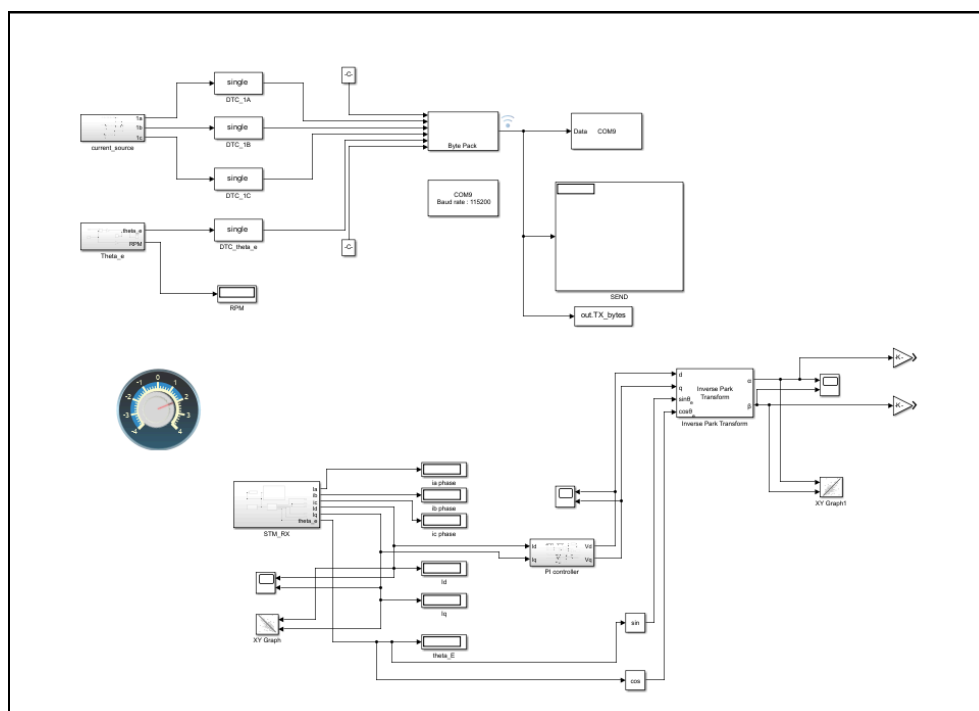
The system is designed to emulate real-time BLDC/PMSM motor operation by generating simulated:

- Three-phase current measurements (I_a , I_b , I_c)
- Electrical rotor angle (θ_e)
- Dynamic torque and RPM responses

These parameters are continuously adjusted based on:

- Required RPM setpoint
- Torque demand
- PID controller response
- Simulated motor load conditions

This creates a realistic environment for testing the hardware design of our FOC speed controller without requiring physical motor hardware while also helping with its firmware.



Complete HIL model for Field Oriented Control Implementation on Simulink

System workflow:

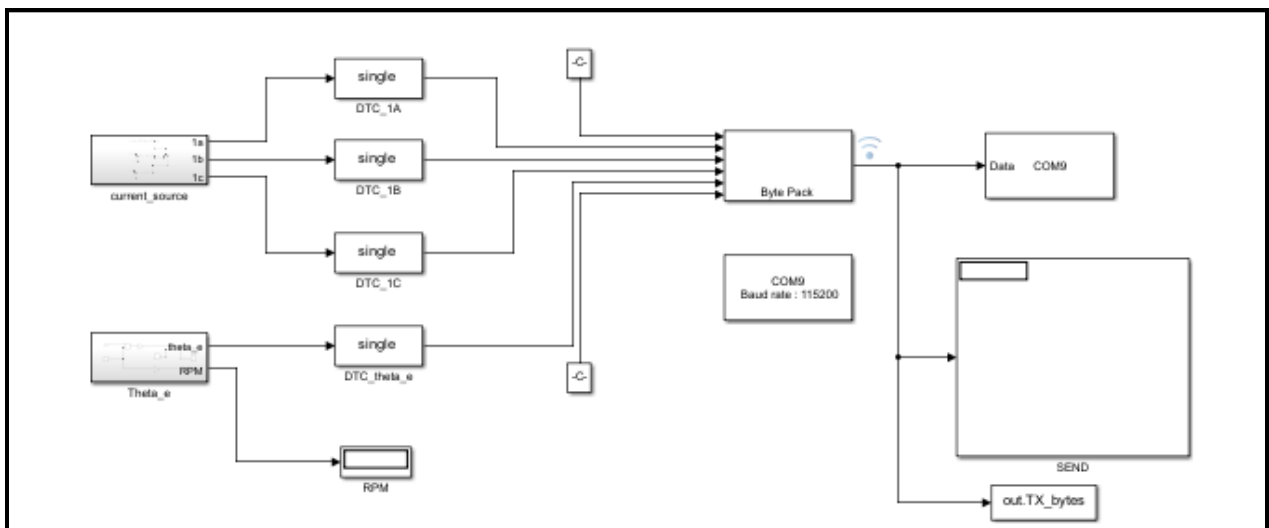
Simulink Motor Model

Simulink acts as the virtual motor and load system:

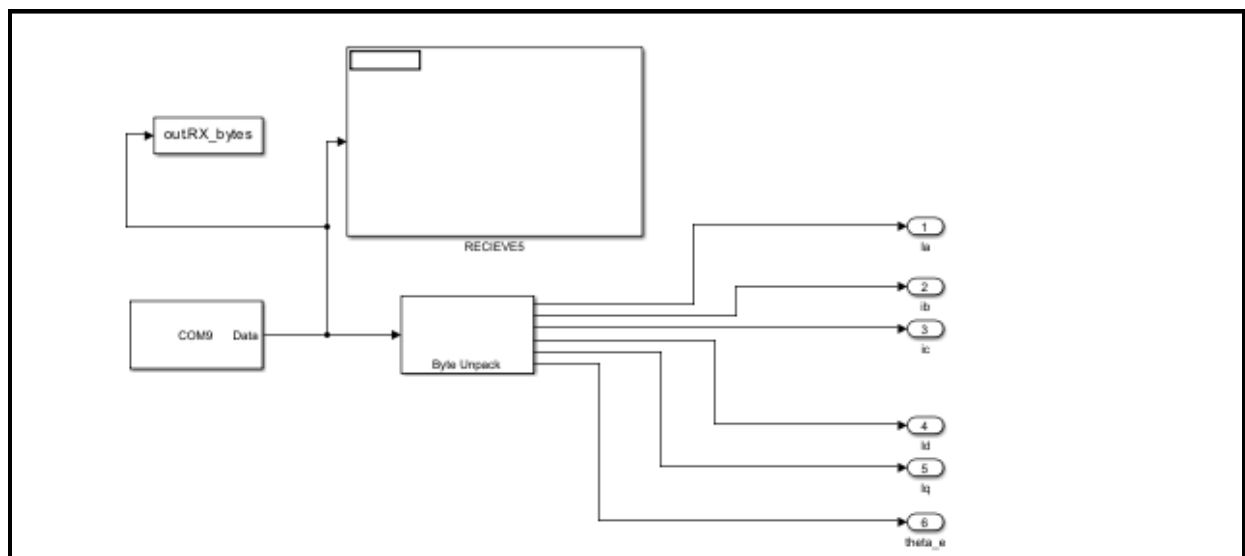
- Generates phase current samples
- Computes electrical angle from RPM
- Simulates torque-speed behavior
- Updates values dynamically according to control inputs

Data Preparation:

- Variables converted to **single-precision floating point**
- Serialized into byte streams
- Sent to STM32G4 over UART/Serial COM



Sender block



Reciever block

2. STM32G431 Embedded Controller

The STM32G431 performs core FOC mathematical operations:

Processing Steps:

- UART data reception
- Incoming byte stream unpacking into single-precision current and electrical angle variables (I_a , I_b , I_c , θ_e)
- Real-time reconstruction of motor state variables
- **Clarke Transformation ($abc \rightarrow \alpha\beta$ frame)**
- **Park Transformation ($\alpha\beta \rightarrow dq$ rotating frame)**
- Extraction of direct-axis current (I_d) for flux control
- Extraction of quadrature-axis current (I_q) for torque control
- Repackaging processed dq values into byte streams
- **UART transmission back to Simulink for closed-loop PID control, monitoring, and visualization**
- Byte unpacking / decoding
- Clarke Transformation
- Park Transformation
- I_d and I_q current extraction
- Returns processed values to Simulink for monitoring

Returned Variables: Direct-axis current (I_d), Quadrature-axis current (I_q) , Control loop outputs

STM32G431CBU6 is a mainstream Arm Cortex-M4 32-bit MCU with DSP and FPU instructions, offering 170 DMIPS / 213 CoreMark performance, high-speed ADCs, and advanced control timers. It is optimized for motor control, digital power conversion, and closed-loop control applications such as Field-Oriented Control (FOC).



3. Closed-Loop Control in Simulink

Returned I_d and I_q values are used in:

- PID torque controller
- Speed controller
- Torque reference tracking
- After controller error correction applied :
 - Inverse Park Transform applied
 - Produces V_α , V_β

Mathematical Relations

Relationship Between Electrical Speed and Theta in RPM

The connection between electrical speed and theta in RPM is essential for generating realistic values of electrical angles without a physical load. The accuracy of this relationship can be demonstrated through the X/Y plot of (i_d) and (i_q), which results in an elliptical shape.

Electrical speed:

$$\omega_e = p \frac{2\pi RPM}{60}$$

Electrical angle:

$$\theta_e = \int \omega_e dt$$

Clarke:

$$I_\alpha = I_a$$

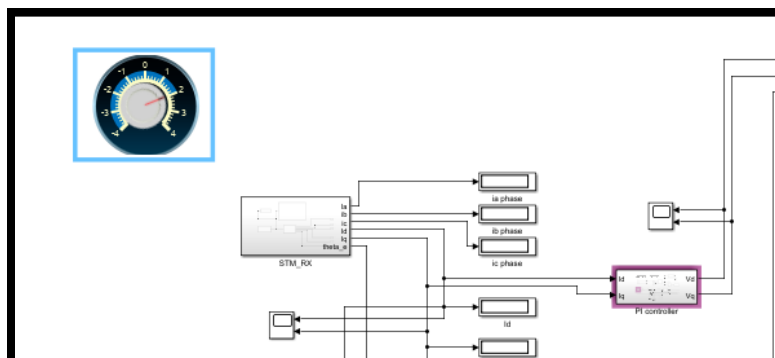
$$I_\beta = -\frac{2I_b + I_a}{\sqrt{3}}$$

Park:

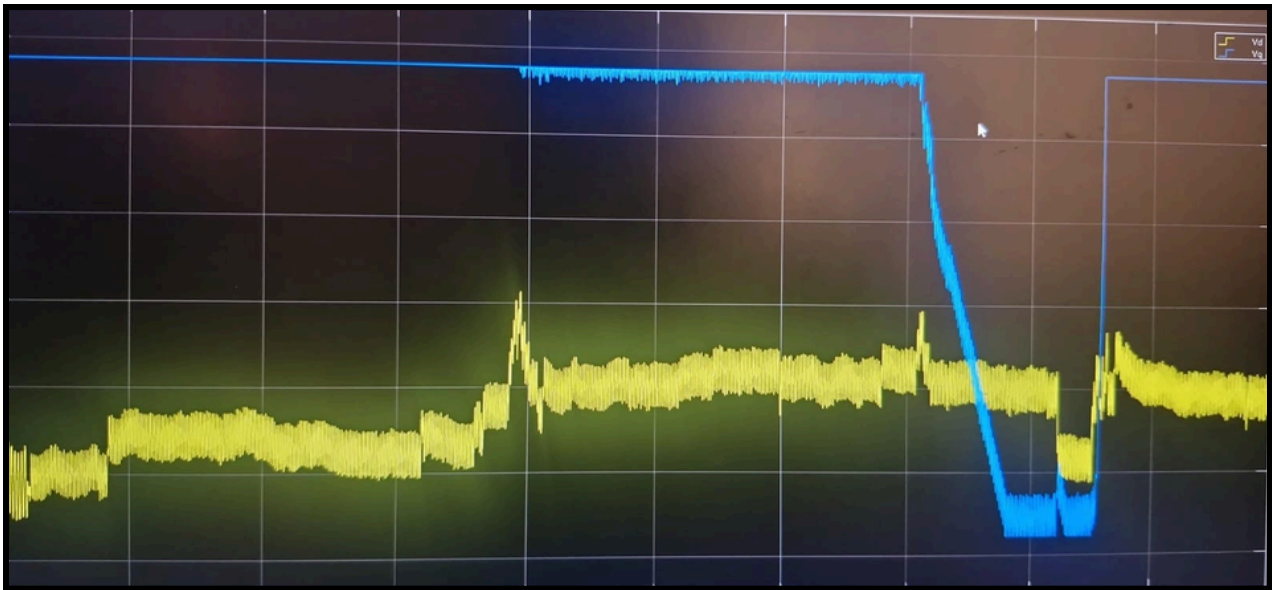
$$I_q = I_\alpha \cos \theta_e - I_\beta \sin \theta_e$$

$$I_d = I_\alpha \sin \theta_e + I_\beta \cos \theta_e$$

The graph can be observed in the accompanying video. **Since the values of I_d and I_q , calculated and transmitted by the STM32G431 microcontroller unit (MCU), have been mathematically verified, we will now proceed with PI tuning to optimize our desired torque output.**



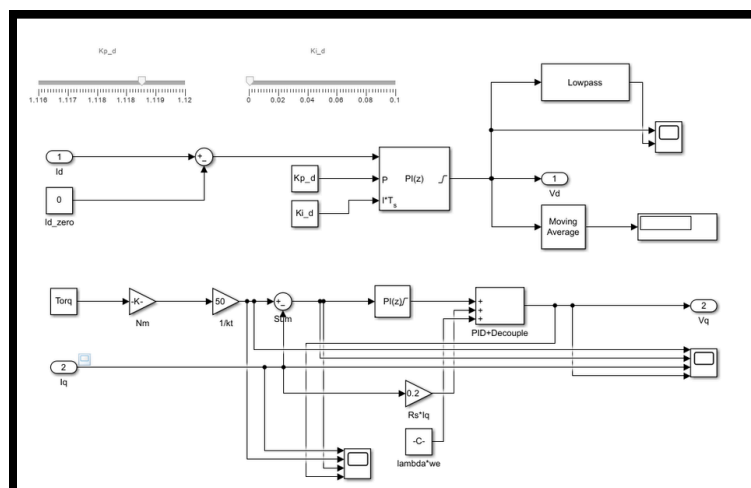
Error Correction & PI Parameter Tuning



Vd (yellow) and Vq(blue) output at max torque without PI block.



Vd and Vq output after tuned PI output for minimal and maximum torque output values.



Complete PI block with decoupling gains

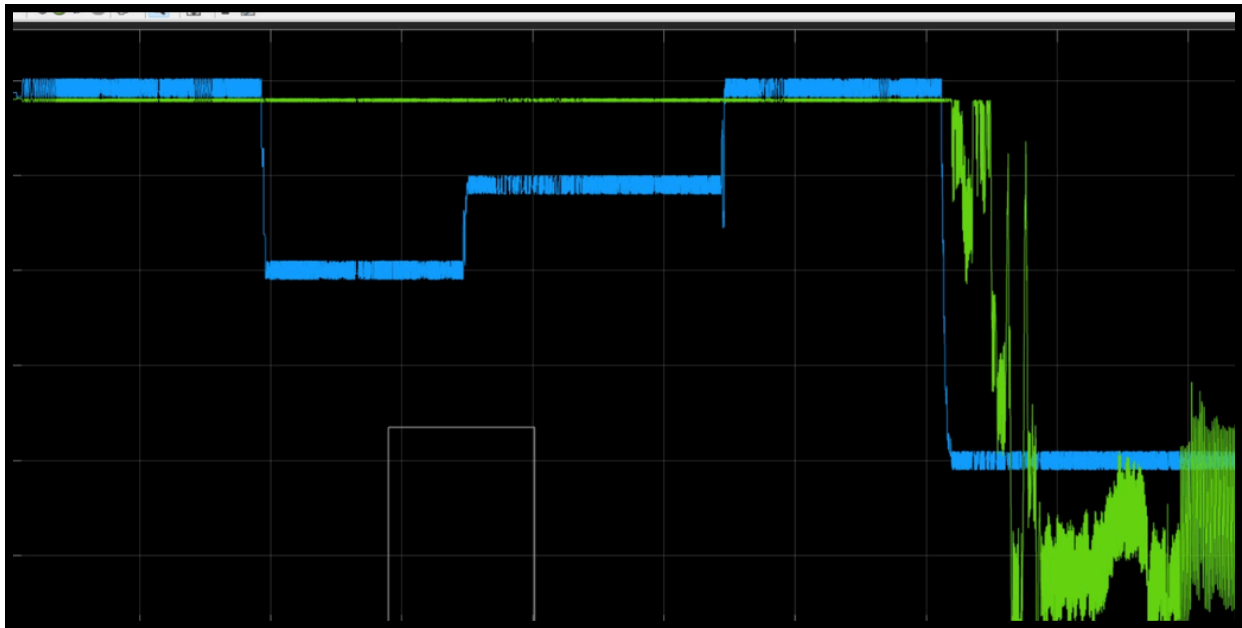
d-axis:

$$V_d = K_{pd}e_d + K_{id} \int e_d dt - \omega_e L_q I_q$$

q-axis:

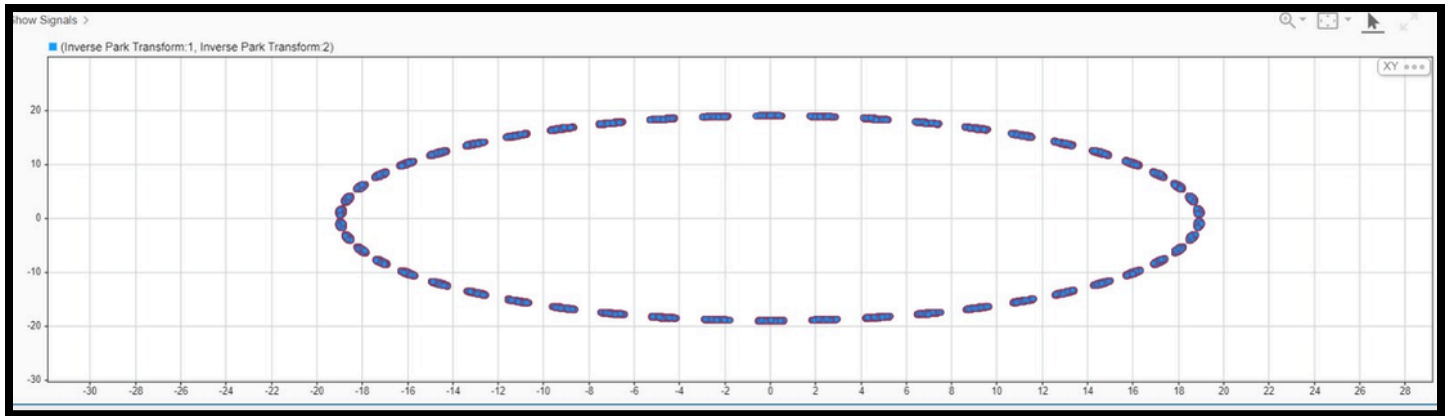
$$V_q = K_{pq}e_q + K_{iq} \int e_q dt + \omega_e (L_d I_d + \lambda)$$

I also included the decoupling terms that are contingent upon the motor's construction. For this simulation, I utilized the characteristics of the T200 thruster from Blue Robotics, which is an industrial-grade thruster designed for underwater applications.



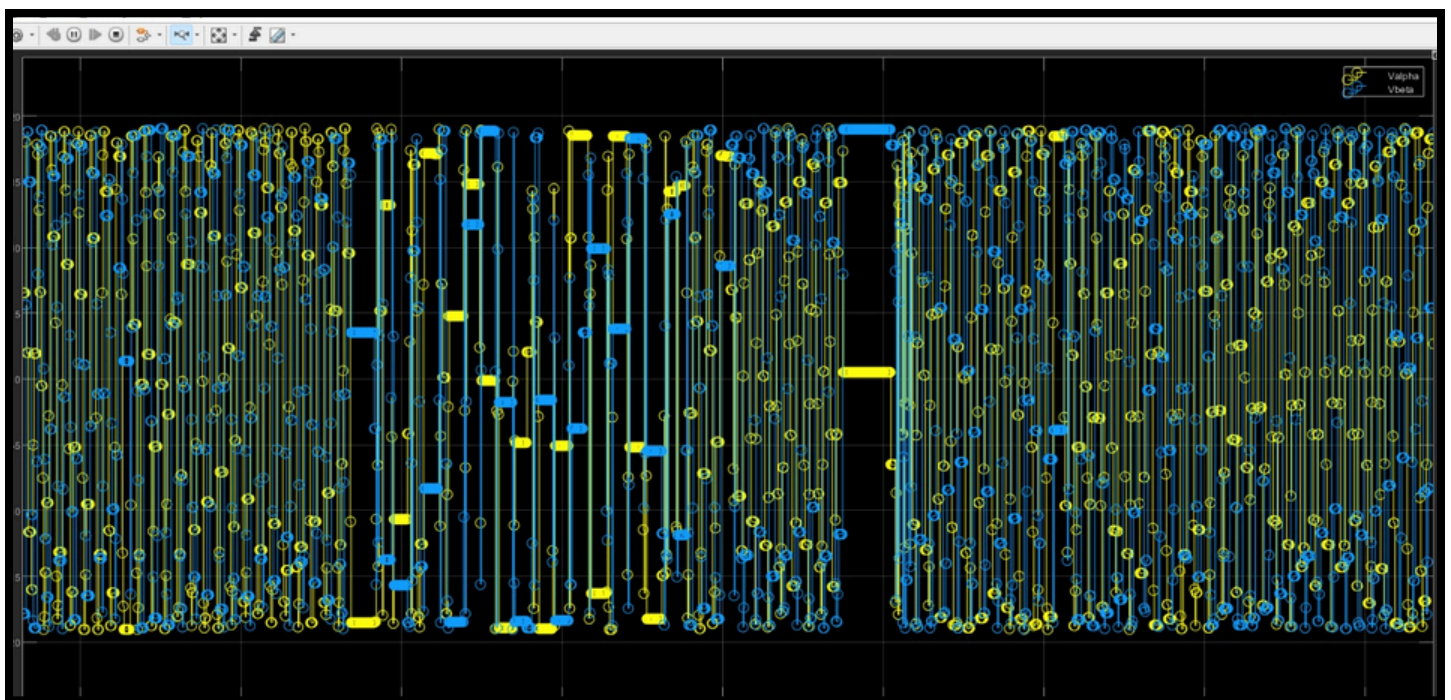
One of the most impressive graphs produced via this project : **The blue trace represents the q-axis current error (difference between the commanded I_q reference derived from torque input and the measured I_q feedback from the STM32), while the green trace shows the PI controller output voltage V_q , including decoupling terms.** As torque commands step through different values, the controller adjusts V_q to minimize current error and maintain closed-loop stability

Final X/Y Plot Following Inverse Park Transformation



This plot confirms the accuracy of the resulting V_α and V_β . Additionally, it validates the error correction achieved through PI tuning.

Finally here is the plot of V_α and V_β with time .



Is this verification sufficient?

In the Hardware-in-the-Loop (HIL) phase, confirming the generated **V α** (V_a) and **V β** (V_β) outputs is sufficient to verify the proper operation of the Field-Oriented Control (FOC) model. These two variables encapsulate the entire synthesized stator voltage vector within the stationary reference frame. **After Inverse Park transformation , all control actions (PI controllers , current sensing , Torque commands, rotor electrical speed) are condensed into this equation :**

$$\begin{aligned}\vec{V}_{ref} &= V_\alpha + jV_\beta \\ V_{ref} &= \sqrt{V_\alpha^2 + V_\beta^2} \\ \theta &= \tan^{-1}\left(\frac{V_\beta}{V_\alpha}\right)\end{aligned}$$

Using these values, Space Vector PWM timing can be reconstructed:

$$\begin{aligned}T_1 &= \frac{\sqrt{3}T_s V_{ref}}{V_{dc}} \sin\left(\frac{\pi}{3} - \theta\right) \\ T_2 &= \frac{\sqrt{3}T_s V_{ref}}{V_{dc}} \sin(\theta) \\ T_0 &= T_s - T_1 - T_2\end{aligned}$$

and for a single sector , the duty cycles calculated are :

$$\begin{aligned}T_a &= \frac{T_1 + T_2 + T_0/2}{T_s} \\ T_b &= \frac{T_2 + T_0/2}{T_s} \\ T_c &= \frac{T_0/2}{T_s}\end{aligned}$$

Consequently, the resulting duty cycle is intrinsically dependent on the **V α** (V_a) and **V β** (V_β) values we have accurately generated in real-time using the STM32 and the simulated input for our desired torque. **By checking V α** (V_a) and **V β** (V_β), the project creates a control core that works with any hardware. This core can move easily between different embedded systems without needing to change the main FOC algorithm. These outputs define the reference voltage vector, so they can work with various microcontrollers, PWM parts, and gate drivers.

Moreover, they help in making real-time plots to study efficiency and find any problems in a new closed-loop algorithm on a different hardware. In Future , I will be adding blocks of DRV series gate drivers and motor models for simulation and testing the hardware of my own closed loop speed controller.